Active Record Migrations

Migrations are a feature of Active Record that allows you to evolve your database schema over time. Rather than write schema modifications in pure SQL, migrations allow you to use an easy Ruby DSL to describe changes to your tables.

1 Migration Overview

Migrations are a convenient way to alter your database schema over time in a consistent and easy way. They use a Ruby DSL so that you don't have to write SQL by hand, allowing your schema and changes to be database independent.

You can think of each migration as being a new 'version' of the database. A schema starts off with nothing in it, and each migration modifies it to add or remove tables, columns, or entries. Active Record knows how to update your schema along this timeline, bringing it from whatever point it is in the history to the latest version. Active Record will also update your `db/schema.rb` file to match the up-to-date structure of your database.

Here's an example of a migration:

```
class CreateProducts < ActiveRecord::Migration
  def change
    create_table :products do |t|
      t.string :name
      t.text :description

      t.timestamps null: false
    end
  end
end
```

This migration adds a table called `products` with a string column called `name` and a text column called `description`. A primary key column called `id` will also be added implicitly, as it's the default primary key for all Active Record models. The `timestamps` macro adds two columns, `created_at`and `updated_at`. These special columns are automatically managed by Active Record if they exist.

Note that we define the change that we want to happen moving forward in time. Before this migration is run, there will be no table. After, the table will exist. Active Record knows how to reverse this migration as well: if we roll this migration back, it will remove the table.

On databases that support transactions with statements that change the schema, migrations are wrapped in a transaction. If the database does not support this then when a migration fails the parts of it that succeeded will not be rolled back. You will have to rollback the changes that were made by hand.

There are certain queries that can't run inside a transaction. If your adapter supports DDL transactions you can use `disable_ddl_transaction!` to disable them for a single migration.

If you wish for a migration to do something that Active Record doesn't know how to reverse, you can use `reversible`:

```
class ChangeProductsPrice < ActiveRecord::Migration
  def change
    reversible do |dir|
      change_table :products do |t|
        dir.up   { t.change :price, :string }
        dir.down { t.change :price, :integer }
```